

DOCUMENTATION TECHNIQUE

Backend iot-backend

API Express/TypeScript : ingestion MQTT, historique SQLite, temps réel SSE, authentification, relais vers le service IA, notifications par e-mail.

Projet	Sentinel-X — Workshop EPSI Bac+4, consortium AetherCorp
Pile	Node.js $\geq$ 22.13 (24 conseillé), TypeScript, Express 5, Better Auth, MQTT.js, node:sqlite, Nodemailer
Public	Développeurs backend, équipe infrastructure et jury technique
Version	Octobre 2026 — état du dépôt à la date de génération

Sommaire

- 01** Rôle dans la plateforme
Point d'entrée unique du dashboard

- 02** Architecture
Clean architecture, couches et règle de dépendance

- 03** Flux de données
Télémétrie, IA, alertes

- 04** Modèle de données
Bases SQLite, tables, rétention

- 05** API HTTP
Routes, contrats, codes d'erreur

- 06** Temps réel et vidéo
SSE, relais MJPEG

- 07** Intégration du service IA
File ordonnée, statut, reconnaissance faciale

- 08** Notifications
Détection d'alertes, anti-spam, transport mail

- 09** Sécurité
Authentification, limites, proxy, secrets

- 10** Configuration et déploiement
Variables, Docker, scripts

- 11** Tests et qualité
Stratégie et résultats

1 Rôle dans la plateforme

`iot-backend` est le **seul point d'entrée du navigateur**. Il reçoit la télémétrie du boîtier via le broker MQTT, la valide, l'enregistre dans SQLite et la diffuse en temps réel au dashboard. Il orchestre aussi l'analyse par le service IA et relaie la vidéo de la webcam. Le navigateur ne contacte jamais directement MQTT ni FastAPI : toutes les routes passent par une session authentifiée.

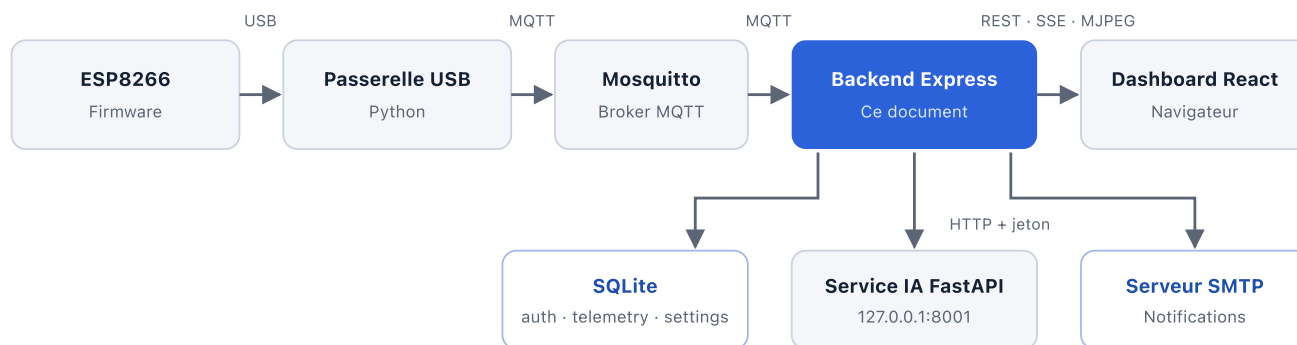


Figure 1. Position du backend dans Sentinel-X et ses dépendances externes.

2 Architecture

Le code suit la **clean architecture**. Les règles métier ne dépendent d'aucun framework, et les dépendances pointent toujours vers le centre : `domain` n'importe rien, `application` n'importe que `domain`, et `presentation` n'importe jamais `infrastructure`. Seul `main.ts` connaît les implémentations concrètes.

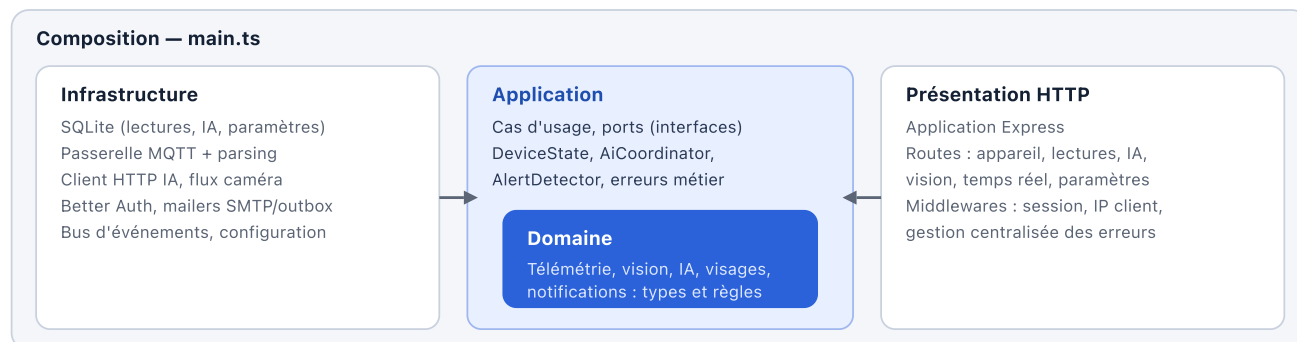


Figure 2. Couches du backend : l'infrastructure et la présentation dépendent de l'application, qui dépend du domaine.

Couche	Contenu principal (src/)
Domaine	<code>domain/telemetry.ts</code> (mesures, indicateurs du boîtier, règle d'horodatage), <code>vision.ts</code> , <code>ai.ts</code> , <code>faces.ts</code> , <code>notification.ts</code> (types d'alertes, délais, validation d'e-mail)
Application	<code>ports.ts</code> (ReadingRepository, DeviceGateway, AiGateway, AiPredictionRepository, LiveEvents, CameraFeed, SessionVerifier, NotificationSettingsRepository, Mailer, Clock) ; cas d'usage <code>use-cases/</code> ; <code>device-state.ts</code> , <code>ai-coordinator.ts</code> , <code>alert-detector.ts</code> , <code>face-enrollment.ts</code> , <code>errors.ts</code>

Couche	Contenu principal (src/)
Infrastructure	<code>persistence/</code> (SQLite), <code>messaging/</code> (client MQTT, parsing), <code>ai/</code> (client HTTP FastAPI, contrat JSON), <code>auth/</code> (Better Auth), <code>camera/</code> , <code>mail/</code> , <code>events/</code> , <code>config.ts</code>
Présentation	<code>presentation/http/app.ts</code> , <code>routes/</code> , <code>middleware/</code>
Outils	<code>main.ts</code> (composition), <code>cli/create-user.ts</code> , <code>tools/simulator.ts</code> (simulateur MQTT et caméra factice)

Cas d'usage	Responsabilité
<code>RecordTelemetry</code>	Horodater, enregistrer, mettre à jour l'état courant, publier l'événement <code>reading</code>
<code>RecordDetection</code>	Mémoriser une détection vision externe et publier <code>vision</code>
<code>GetDeviceStatus</code> / <code>GetReadingHistory</code> / <code>GetRecentDetections</code>	Lectures pour l'API (historique borné à 500 lignes)
<code>SendDeviceCommand</code>	Valider <code>ON/OFF</code> et publier sur le topic de commande
<code>CheckDeviceHeartbeat</code> / <code>NotifyAlert</code>	Détecter le silence du boîtier et envoyer les alertes par e-mail
<code>Get/UpdateNotificationSettings</code> , <code>SendTestNotification</code>	Paramètres d'alerte et e-mail de test
<code>PruneReadingHistory</code>	Purge horaire au-delà de la rétention

3 Flux de données

3.1 Ingestion de la télémetrie

- 1 `MqttDeviceGateway` reçoit un message sur `esp8266/donnees`. `parseTelemetryMessage` le valide : nombres finis, `presence` booléen ou 0/1, `ts` en secondes Unix ou `timestamp` ISO, indicateurs booléens du boîtier, `source` `live` ou `simulation`. Un message invalide est rejeté en entier.
- 2 `RecordTelemetry` détermine l'heure d'enregistrement avec `resolveRecordedAt` : l'heure de mesure est conservée si elle est plausible (au plus 60 s dans le futur, pas plus vieille que la rétention), sinon l'heure de réception est utilisée.
- 3 La mesure est écrite dans la table `readings` avec sa source et ses indicateurs. `DeviceState` ne prend en compte que les mesures **physiques** les plus récentes : une mesure rejouée ou simulée ne remplace jamais l'état courant.
- 4 L'événement `reading` est publié sur le bus interne. Le flux SSE le diffuse, l'`AiCoordinator` le place dans sa file et l'`AlertDetector` l'évalue.

3.2 Analyse IA

L'`AiCoordinator` envoie les mesures récentes (moins de 30 s, au plus 5 s dans le futur, ordre chronologique respecté par source) à `POST /analyze`, **une à la fois** pour préserver l'ordre des features temporelles. La file contient au plus 32 mesures ; en surcharge, la plus ancienne est abandonnée et `dropped_samples` est incrémenté. Les indicateurs de préchauffage et de validité excluent les canaux non fiables du lot envoyé. Le résultat validé est enregistré dans `ai_predictions`, puis publié en SSE (`ai`). Un sondage de `/status` toutes les 2 s alimente `ai-status`, même sans nouvelle mesure.

À SAVOIR

MQTT n'attend jamais l'IA. Si FastAPI est arrêté ou lent, le statut passe hors ligne, les mesures continuent d'être enregistrées et le dernier résultat reste consultable. Aucun résultat NORMAL ou SAFE n'est inventé.

3.3 Alertes

L'`AlertDetector` transforme les événements en alertes uniquement sur **changement d'état** :

- intrusion : passage du PIR à « présence », pour une mesure physique récente (moins de 60 s) ;
- visage inconnu : détection contenant un visage sans nom ;
- boîtier hors ligne : aucune mesure physique depuis 30 s (vérification toutes les 10 s), une seule alerte par silence.

Les mesures et détections simulées ne déclenchent jamais de notification réelle.

4 Modèle de données

Le stockage utilise le module natif `node:sqlite` (aucune dépendance native à compiler), en mode WAL. Trois fichiers, configurables et ignorés par Git :

Fichier (défaut)	Tables	Contenu
<code>data/auth.db</code>	<code>user</code> , <code>session</code> , <code>account</code> , <code>verification</code>	Comptes et sessions Better Auth (migrations automatiques au démarrage, mots de passe hachés en scrypt)
<code>data/telemetry.db</code>	<code>readings</code>	<code>id</code> , <code>recorded_at</code> (ms), <code>temperature</code> , <code>humidity</code> , <code>gas</code> , <code>presence</code> , <code>source</code> (<code>live/simulation</code> , NULL pour l'historique antérieur à la migration), <code>device_flags</code> (JSON) ; index sur <code>recorded_at</code>
	<code>ai_predictions</code>	<code>id</code> , <code>event_key</code> unique (déduplication), <code>recorded_at</code> , <code>payload</code> (prédiction JSON complète)
<code>data/settings.db</code>	<code>notification_settings</code>	Ligne unique : adresse e-mail, activation, types d'alertes (JSON)

Les lectures et les prédictions IA partagent la même rétention (`READINGS_RETENTION_DAYS`, 7 jours par défaut), appliquée au démarrage puis toutes les heures. La provenance `source` permet d'exporter uniquement les mesures réelles pour l'entraînement du modèle IA.

5 API HTTP

Better Auth est monté sur `/api/auth/*`. `GET /api/health` est public. Toutes les autres routes exigent une session (cookie) et renvoient `401` sinon.

Méthode et route	Rôle
<code>GET /api/status</code>	État MQTT, dernière mesure physique et son heure
<code>GET /api/readings?limit=N</code>	Historique, plus récent d'abord ($1 \leq N \leq 500$)
<code>GET /api/vision</code>	Détections vision externes récentes
<code>GET /api/stream</code>	Flux SSE : <code>reading</code> , <code>vision</code> , <code>ai</code> , <code>ai-status</code>
<code>GET /api/camera/stream</code>	Relais MJPEG authentifié (alias <code>/api/ai/vision/stream</code>)
<code>POST /api/action</code>	<code>{"ordre":"ON" "OFF"}</code> , publié sur <code>esp8266/led</code> — <code>202</code>
<code>GET /api/ai/status</code> , <code>/latest</code> , <code>/history?limit=N</code>	Disponibilité, dernière prédiction (priorité au réel pendant 30 s), historique
<code>POST /api/ai/vision/start / stop</code> , <code>GET /api/ai/vision/status</code>	Pilotage de la webcam du serveur
<code>GET /api/ai/vision/faces/status</code> , <code>/latest</code> , <code>/history</code>	Catalogue facial, reconnaissances courantes, 20 derniers événements
<code>POST /api/ai/vision/faces/reload</code>	Rechargement atomique des photos de référence
<code>POST /api/ai/vision/faces/enroll</code>	Ajout de 1 à 5 photos pour une identité (JSON base64, 21 Mio max)
<code>GET / PUT /api/settings/notifications</code>	Lecture et mise à jour des paramètres d'alerte
<code>POST /api/settings/notifications/test</code>	E-mail de test (au plus toutes les 30 s) — <code>202</code>

Exemple : télémetrie publiée par la passerelle

```
{ "source": "live", "device": "sentinel-x-01", "ts": 1791370000.5,
  "temperature": 24.4, "humidity": 61.1, "gas": 40, "presence": true,
  "climateValid": true, "gasReady": true, "pirReady": true, "gasAlert": false,
  "alarmActive": true, "ledRed": true, "ledOrange": false, "ledGreen": false }
```

Codes d'erreur

Un gestionnaire central traduit les erreurs applicatives. Les réponses ont toujours la forme `{ "message": "..."} .`

Code	Cause
<code>400</code>	Requête invalide (validation métier, JSON mal formé)
<code>401</code> / <code>403</code>	Session absente ou expirée / origine non autorisée par Better Auth

Code	Cause
413 / 422	Corps trop volumineux / photos d'enrôlement inexploitable
429	Trop de tentatives (connexion, e-mail de test)
502	Commande ou e-mail non remis par le broker ou le serveur SMTP ; flux caméra injoignable
503	Broker déconnecté, transport mail ou service IA/caméra indisponible

6 Temps réel et vidéo

GET `/api/stream` ouvre un flux **Server-Sent Events** (`text/event-stream`). Chaque événement transporte l'objet JSON concerné : une lecture enregistrée (avec son `id`), une détection, une prédiction IA persistée ou le statut IA. Un commentaire de maintien est envoyé toutes les 15 s, et le flux est fermé quand la session expire : le client revient alors à l'écran de connexion. L'en-tête `X-Accel-Buffering: no` désactive la mise en tampon de nginx.

La vidéo ne passe jamais par SSE. `/api/camera/stream` relaie en flux continu le MJPEG du service IA, ou d'un service externe défini par `VISION_STREAM_URL`. Le relais préserve le `Content-Type`, ajoute le jeton du service IA côté serveur et annule la requête amont quand le navigateur se déconnecte.

7 Intégration du service IA

- **Contrat validé** : `infrastructure/ai/ai-contract.ts` vérifie chaque réponse JSON de FastAPI avant persistance ou diffusion.
- **Authentification de service** : si `AI_SERVICE_TOKEN` est défini, il est envoyé en `Authorization: Bearer`. Il n'est jamais transmis au navigateur.
- **Délais** : 5 s par appel (`AI_TIMEOUT_MS`) ; 60 s pour le rechargement et l'enrôlement des visages.
- **Source de la caméra** : `VISION_STREAM_URL` explicite prioritaire ; absent, le backend relaie `AI_SERVICE_URL/stream.mjpg` ; vide, la vidéo est désactivée.
- **Séparation réel / simulation** : les prédictions conservent leur source ; une analyse simulée ne remplace pas l'état canonique de la webcam ni une analyse réelle récente.
- **Enrôlement facial** : seule cette route accepte un corps JSON de 21 Mio, **après** vérification de la session. Limites : 5 photos, 5 Mio par photo et 15 Mio au total une fois décodées. Les noms sont normalisés et filtrés (séparateurs de chemin, noms réservés).

8 Notifications

Règle	Valeur
Types d'alertes	<code>intrusion</code> , <code>unknown-face</code> , <code>device-offline</code> (activables individuellement)
Anti-spam	un e-mail par type toutes les 5 minutes au plus

Règle	Valeur
Fraîcheur	un événement de plus de 60 s ne déclenche pas d'alerte
E-mail de test	au plus toutes les 30 s
Validation	adresse ≤ 254 caractères, sans espace ni retour ligne (anti-injection d'en-tête) ; adresse obligatoire pour activer

Le port `Mailer` a trois implémentations, choisies au démarrage : `Smtplib` si `SMTP_HOST` est défini (STARTTLS exigé hors TLS implicite), `OutboxMailer` si `MAIL_OUTBOX_DIR` est défini (fichiers `.eml`, mode test), sinon `UnconfiguredMailer` (aucun envoi, signalé dans l'interface).

9 Sécurité

- **Authentification** : Better Auth, e-mail et mot de passe (12 caractères minimum), cookie de session `HttpOnly`, sessions de 8 h renouvelées toutes les heures. L'inscription publique est désactivée ; les comptes se créent en ligne de commande.
- **Limitation de débit** : 100 requêtes d'authentification par minute et par IP, 5 tentatives de connexion par minute. L'IP est lue dans `X-Forwarded-For`, que le backend réécrit avec l'IP résolue selon `TRUST_PROXY` : un client ne peut pas la falsifier pour contourner le blocage.
- **Origines** : CORS et origines de confiance limitées à `FRONTEND_ORIGINS` ; une connexion depuis une autre origine est refusée (403).
- **Taille des requêtes** : 16 Kio par défaut ; 21 Mio uniquement pour l'enrôlement facial authentifié.
- **Secrets** : `BETTER_AUTH_SECRET` obligatoire (≥ 32 caractères, refus de démarrer sinon) ; aucun secret dans le dépôt (`.env` ignorés, contrôle `scripts/check-repository.py`).
- **Traçabilité** : chaque commande LED et chaque modification des paramètres sont journalisées avec l'e-mail de l'opérateur.
- **Transport** : en production, le backend sert lui-même HTTPS (`TLS_CERT_FILE`, TLS 1.2 minimum) derrière nginx, se connecte au broker en MQTTS avec un compte dédié et appelle l'IA en HTTPS, en vérifiant tous les certificats avec la CA locale (`NODE_EXTRA_CA_CERTS`).
- **Secrets en fichiers** : `BETTER_AUTH_SECRET_FILE`, `AI_SERVICE_TOKEN_FILE`, `MQTT_PASSWORD_FILE` et `SMTP_PASSWORD_FILE` sont prioritaires sur les variables en clair.
- **Conteneur** : utilisateur non-root, système de fichiers en lecture seule, capacités Linux retirées, `no-new-privileges`, API non publiée (accessible seulement via nginx).

10 Configuration et déploiement

Variable	Défaut	Rôle
<code>PORT</code>	3001	Port HTTP
<code>MQTT_URL</code>	<code>mqtt://127.0.0.1:1883</code>	Broker
<code>MQTT_TELEMETRY_TOPIC</code> / <code>_COMMAND_TOPIC</code> / <code>_VISION_TOPIC</code>	<code>esp8266/donnees</code> · <code>esp8266/led</code> · <code>sentinel/vision</code>	Topics

Variable	Défaut	Rôle
AI_SERVICE_URL / AI_SERVICE_TOKEN	http://127.0.0.1:8001 / vide	Service IA (vide : IA désactivée)
AI_TIMEOUT_MS / AI_POLL_MS	5000 / 2000	Délai d'appel, période de sondage
VISION_STREAM_URL	absent	Flux caméra explicite ; vide = désactivé
TELEMETRY_DATABASE_PATH · AUTH_DATABASE_PATH · SETTINGS_DATABASE_PATH	data/*.db	Bases SQLite
READINGS_RETENTION_DAYS	7	Rétention des lectures et prédictions
BETTER_AUTH_SECRET / BETTER_AUTH_URL	obligatoire / 1 ^{re} origine	Secret de session, URL publique
FRONTEND_ORIGINS	origines Vite locales	Origines autorisées (liste)
TRUST_PROXY	loopback	Proxys autorisés à transmettre l'IP client
SMTP_HOST, SMTP_PORT, SMTP_SECURE, SMTP_USER, SMTP_PASSWORD, MAIL_FROM	587, false	Envoi des notifications
MAIL_OUTBOX_DIR	vide	Mode test : e-mails écrits en .eml
TLS_CERT_FILE / TLS_KEY_FILE	vide	Servir en HTTPS (production)
MQTT_USERNAME / MQTT_PASSWORD(_FILE)	vide	Compte du broker
PUBLIC_HOSTS / PUBLIC_HTTPS_PORT	vide / 443	Origines HTTPS déduites si FRONTEND_ORIGINS est absent
INITIAL_ADMIN_EMAIL / INITIAL_ADMIN_PASSWORD_FILE	vide	Administrateur créé si la base de comptes est vide

Scripts npm

```

npm run dev          # tsx watch src/main.ts (rechargement automatique)
npm run build        # compilation TypeScript vers dist/
npm start            # node dist/main.js
npm test             # tests (runner natif de Node via tsx)
npm run typecheck    # vérification de types
npm run user:create -- operateur@exemple.fr "Nom"
npm run simulate     # simulateur MQTT + caméra factice (source: simulation)

```

Docker : image multi-étapes node:24-slim, healthcheck HTTPS sur /api/health. docker-compose.yml déploie toute la plateforme : un service `init` génère la CA locale, les certificats et les secrets, puis nginx, le backend, Mosquitto (TLS uniquement), le service IA et la passerelle USB démarrent. Le backend est relié à nginx sur le réseau `edge` (`TRUST_PROXY` limité à ce sous-réseau) et au broker et à l'IA sur le réseau interne `core`, sans Internet. Le volume `backend-data` conserve les bases. Voir le guide de déploiement.

11 Tests et qualité

Les tests s'exécutent avec le runner natif de Node (`node:test` via `tsx`), sans broker, sans base persistante ni service IA réel :

- **domaine** : horodatage plausible, commandes, règles de notification ;
- **application** : cas d'usage, état de l'appareil, coordinateur IA, détecteur d'alertes et enrôlement facial, avec des faux adaptateurs en mémoire ;
- **infrastructure** : parsing MQTT, dépôts SQLite en mémoire, client IA, contrats faciaux ;
- **présentation** : routes IA et enrôlement (authentification exigée avant traitement, limites de taille).

Vérification (état du dépôt à la génération)	Résultat
<code>npm test</code>	84/84 réussis
<code>npm run typecheck</code> et <code>npm run build</code>	sans erreur

Le job *backend* de la CI GitHub Actions rejoue ces trois étapes sur Node 24 à chaque push. Des scripts d'intégration complémentaires (`scripts/check-ai-integration.mjs`, `check-face-enrollment.mjs`, `check-camera-integration.mjs`) démarrent des processus réels isolés, avec bases et ports temporaires.